

Révision

Partie 1 : Introduction à Spring & Spring Boot

1. C'est quoi Spring ?

- C'est un cadre de développement
- Contient des «bonnes pratiques»
- Permet de se focaliser sur le métier

2. C'est quoi SpringBoot ?

- Extension de Spring pour simplifier la configuration.
- Il diminue énormément le temps de développement et augmente la productivité

3. Différences Spring vs Spring Boot

Spring	SpringBoot
Beaucoup de configuration manuelle	Configuration simple
Type de livrable : .WAR	Type de livrable : .JAR

4. Avantages de Spring Boot

- **Configuration Simple**

Spring Boot facilite la gestion des configurations en centralisant les configurations dans un seul fichier: **App.properties**



```
1 server.servlet.context-path=/tpfoyer
2 server.port=9090
3 ### DATABASE ###
4 spring.datasource.url=jdbc:mysql://localhost:3306/TpFoyer?createDatabaseIfNotExist=true
5 spring.datasource.username=root
6 spring.datasource.password=
7 ### JPA / HIBERNATE ###
8 spring.jpa.show-sql=true
9 spring.jpa.hibernate.ddl-auto=update
10
```

☐ Spring Boot peut s'expliquer simplement par l'illustration ci-dessous:



Partie 2 : Maven

1. C'est quoi Maven ?

Maven est un outil pour faciliter la construction de projets (Build, Dépendances, ...) et l'automatisation des différentes phases projets (Tests, Packaging, ...). Maven permet donc :

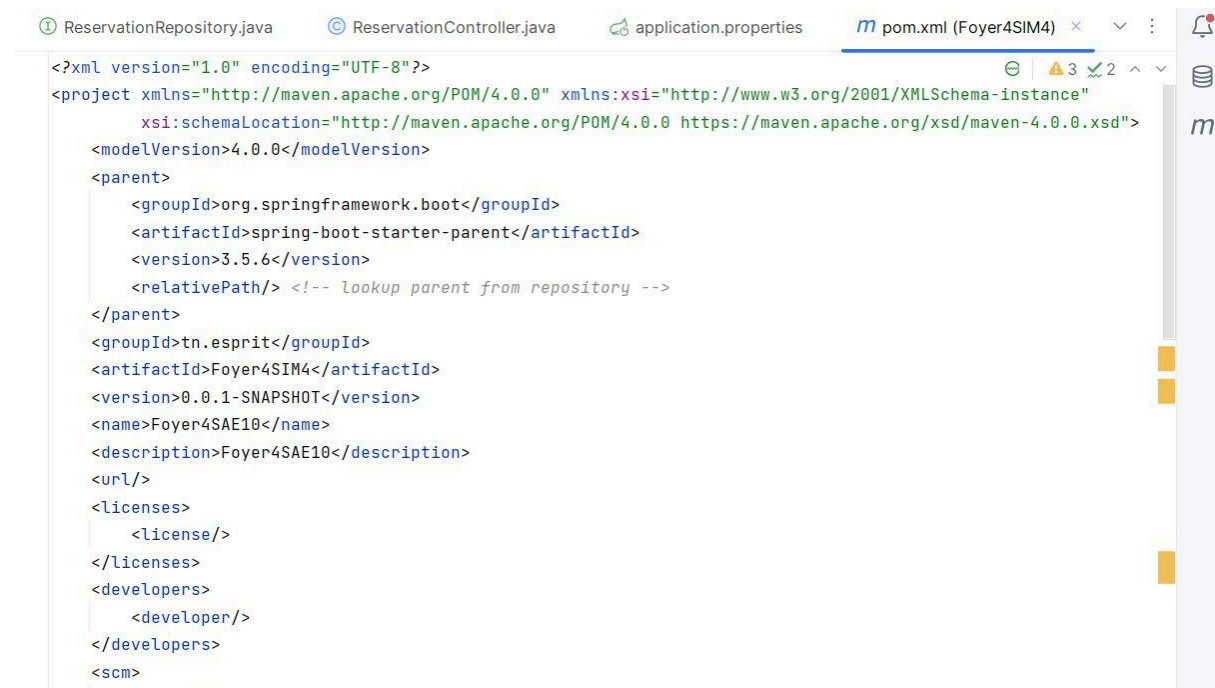
- Gestion des builds (compilation)
- La gestion des dépendances
- Génération des livrables

=> Automatisation de tâches récurrentes => Intégration Continue (DevOps)

2. Fichier de configuration POM.XML

Permet de spécifier les dépendances dont le projet a besoin.

➤ Spécifie l'emplacement de l'artefact du projet (groupId, ArtifactId, version)

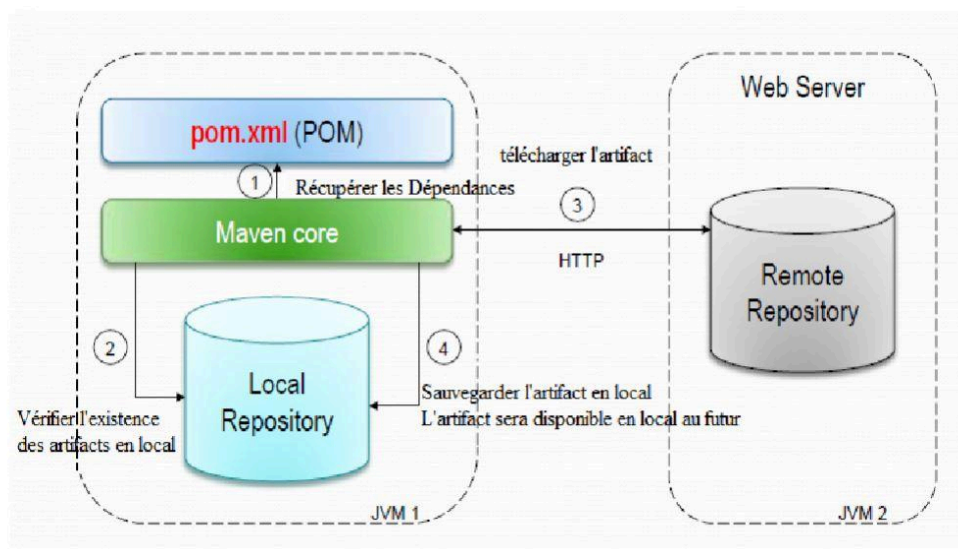


```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.6</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>tn.esprit</groupId>
    <artifactId>Foyer4SIM4</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>Foyer4SAE10</name>
    <description>Foyer4SAE10</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
```

3. Cycle de vie Maven (phases)

1. **validate** – vérifie le projet.
2. **compile** – compile le code Java.
3. **test** – exécute les tests.
4. **package** – génère le **.jar** ou **.war**.
5. **verify** – vérifie la qualité du package.
6. **install** – installe l'artefact dans le repo local.
7. **deploy** – déploie dans un repo distant.

3. Gestion des dépendances et types de dépôts Maven



Local repository : sur la machine (**c:/Users/User/.m2/repository**).

Central repository : repo public de Maven. (<https://mvnrepository.com>)

Remote repository : repo privé (ex : Nexus, Artifactory).

Partie 3 : Les entités JPA

Une entité représente une table dans la base de données. En Spring Boot (JPA), elle est définie avec l'annotation : **@Entity**

```
@Entity
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
public class Chambre {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long idChambre;
    Long numeroChambre;

    @Enumerated(EnumType.STRING)
    TypeChambre typeChambre;
}
```

Chaque entité correspond à une table dans la base de données.

Géré par Lombok

La déclaration d'une clé primaire est obligatoire sur un attribut

Facultatif, sur l'attribut annoté avec @Id. Définit la manière dont la base gère la génération de la clé primaire.

```
public enum TypeChambre {  
    no usages  
    SIMPLE, DOUBLE, TRIPLE  
}
```

Exercice :

// Complétez les annotations manquantes pour cette entité

...

```
public class Etudiant {
```

```
@Id
```

// Clé primaire, stratégie d'auto-increment à compléter

.....

```
private Long id;
```

```
private String nom;
```

```
private String prenom;
```

// Attribut de type énumération à compléter :

```
private Niveau niveau;
```

```
}
```

// Complétez cette énumération avec les niveaux possibles (exemple : DEBUTANT, INTERMEDIAIRE, AVANCE)

```
public enum Niveau {
```

// TODO : compléter les valeurs de l'énumération

```
}
```

💡 Autres annotations utiles en entité

1. Annotations pour la gestion des dates

`@Temporal(TemporalType.DATE)` → pour les dates (jour/mois/année)

2. Annotations pour l'unicité et la contrainte

- `@Column(unique = true)` → rend une colonne unique

3. Autres annotations utiles

- `@Enumerated(EnumType.STRING)` → pour stocker des enums

Partie 4 : Les associations

En JPA, les entités peuvent être liées entre elles grâce à des associations. Il existe 4 types principaux :

One to One(1:1) Unidirectionnelle



```
@Entity
public class Projet {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String sujet;

    @OneToOne
    private Projet_Detail projetDetail;

}
```

```
@Entity
public class Projet_Detail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String description;

    private String technologie;

    private Long cout_provisoire;

    private Date dateDebut;

}
```

One to One(1:1) Bidirectionnelle



<pre>@Entity public class Projet { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; private String sujet; @OneToOne private Projet_Detail projetDetail; }</pre>	<pre>@Entity public class Projet_Detail { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; private String description; private String technologie; private Long cout_provisoire; private Date dateDebut; @OneToOne(mappedBy="projetDetail") private Projet projet; }</pre>
---	--

One to Many(1:*) Unidirectionnelle



<pre>@Entity public class Entreprise { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; private String nom; private String adresse; @OneToMany(cascade = CascadeType.ALL) private Set<Equipe> Equipes; }</pre>	<pre>@Entity public class Equipe { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; private String nom; private String specialite; }</pre>
--	---

One to Many(1:*) Bidirectionnelle

One To Many Bidirectionnelle = Many To One Bidirectionnelle

Many to One(*:1) :Unidirectionnelle



<pre>@Entity public class Equipe { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; private String nom; private String specialite; @ManyToOne(cascade = CascadeType.ALL) Entreprise entreprise; }</pre>	<pre>@Entity public class Entreprise { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; private String nom; private String adresse; }</pre>
--	---

Many to One(*:1) :Bidirectionnelle



<pre>@Entity public class Equipe { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; private String nom; private String specialite; @ManyToOne Entreprise entreprise; }</pre>	<pre>@Entity public class Entreprise { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; private String nom; private String adresse; @OneToMany(cascade = CascadeType.ALL, mappedBy="entreprise") private Set<Equipe> Equipes; }</pre>
---	--

Many to Many(*:*) :Unidirectionnelle



```
@Entity
public class Equipe {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nom;

    private String specialite;

    @ManyToMany(cascade = CascadeType.ALL)
    private Set<Projet> projets;
}
```

```
@Entity
public class Projet {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String sujet;
}
```

Many to Many(*:*) :Bidirectionnelle



```
@Entity
public class Equipe {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nom;

    private String specialite;

    @ManyToMany(cascade = CascadeType.ALL)
    private Set<Projet> projets;
}
```

```
@Entity
public class Projet {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String sujet;

    @ManyToMany(mappedBy="projets", cascade = CascadeType.ALL)
    private Set<Equipe> equipes;
}
```


Partie 5 : Les Repositories+Services+Controllers

- La couche repositories :

La couche Repository est dédiée à l'accès aux données. Elle permet de communiquer avec la base via Spring Data JPA et élimine le besoin d'écrire du SQL manuel.

Type des Repositories:



Implémentation:



- La couche Services :

La couche Service contient la logique métier. Elle sert d'intermédiaire entre les Controllers et les Repositories.

On y trouve généralement :

- Une interface (ex : `IChambreService`) qui définit les signatures des méthodes.
- Une classe d'implémentation (ex : `ChambreServiceImpl`) qui contient la logique et utilise le repository.

Implémentation:

```

public interface IChambreServices {
    1 usage 1 implementation
    Chambre addChambre(Chambre chambre);
    1 usage 1 implementation
    Chambre updateChambre(Chambre chambre);
    1 usage 1 implementation
    void deleteChambre(Long id);
    1 usage 1 implementation
    Chambre getChambreById(Long id);
    1 usage 1 implementation
    List<Chambre> getAllChambres();
}

@Service
public class IChambreServicesImp implements IChambreServices {

    @Autowired
    private ChambreRepository chambreRepository;

    1 usage
    @Override
    public Chambre addChambre(Chambre chambre) { return chambreRepository.save(chambre); }

    1 usage
    @Override
    public Chambre updateChambre(Chambre chambre) { return chambreRepository.save(chambre); }

    1 usage
    @Override
    public void deleteChambre(Long id) { chambreRepository.deleteById(id); }

    1 usage
    @Override
    public Chambre getChambreById(Long id) { return chambreRepository.findById(id).orElse( other: null); }

    1 usage
    @Override
    public List<Chambre> getAllChambres() { return chambreRepository.findAll(); }
}

```

• La couche Controllers :

La couche Controller représente la couche de présentation .

Elle reçoit les requêtes HTTP, appelle la couche Service, et renvoie les réponses au client.

Un Controller porte l'annotation :

- @RestController → indique que c'est un contrôleur REST
- @RequestMapping("/user") → définit le chemin global

Principales méthodes REST :

❖ POST	Permet d'insérer, créer, ajouter une ressources	Pour exposer une méthode d'ajout dans springBoot, on Ajoute : @PostMapping
❖ PUT	Permet de modifier une ressources existantes	Pour exposer une méthode de modif dans springBoot, on Ajoute : @PutMapping
❖ GET	Permet de récupérer une ressource	Pour exposer une méthode d'affichage dans springBoot, on Ajoute : @GetMapping
❖ DELETE	Permet de supprimer une ressource	Pour exposer une méthode de suppression dans springBoot, on Ajoute : @DeleteMapping

Implémentation:

```
@RestController
@AllArgsConstructor
@RequestMapping("/chambres")
public class ChambreController {

    private final IChambreServices chambreServices;

    @PostMapping("/ajouterChambre")
    public Chambre ajouterChambre(@RequestBody Chambre chambre) { return chambreServices.addChambre(chambre); }

    @PutMapping("/modifierChambre")
    public Chambre modifierChambre(@RequestBody Chambre chambre) { return chambreServices.updateChambre(chambre); }

    @DeleteMapping("/supprimerChambre/{id}")
    public void supprimerChambre(@PathVariable Long id) { chambreServices.deleteChambre(id); }

    @GetMapping("/getChambre/{id}")
    public Chambre getChambre(@PathVariable Long id) { return chambreServices.getChambreById(id); }
```

@PathVariable	Sert à prendre une valeur dans l'URL, Par exemple, si je veux supprimer un objet précis je met son ID dans l'URL
@RequestBody	Indique qu'on va insérer un objet avec ses attributs depuis le client (Front, Outil de test..)

Exercice :

1. Repository à compléter

Complétez les parties manquantes dans ce repository sachant que ce repo extends depuis JpaRepository

```
import org.springframework.data.jpa.repository.JpaRepository;
```

// A compléter :

```
public interface ProduitRepo //..... <Produit, Long> {

}
```

2. Service à compléter

// A compléter :

```
public class ProduitServiceImpl implements IProduitService {
```

```
    private final ProduitRepo produitRepo;
```

```
    @Override
```

```
    public Produit addProduit(Produit p) {
```

```
    // A compléter : .....}
```

```

@Override
public void deleteProduit(Long id) {
// A compléter : .....
}
}

```

3. Contrôleur à compléter

Complétez le contrôleur pour exposer les endpoints POST et DELETE :

// A compléter : Ajouter l'annotation de RestController et le mapping global

```
public class ProduitController {
```

```
private final IProduitService produitService;
```

//A compléter @.....("/add")

```
public Produit addProduit(//A compléter .... Produit produit) {
```

```
    return produitServices.addProduit(produit);
}
```

//A compléter @.....("/delete/{id}")

```
public void deleteProduit(//A compléter .... Produit produit) {
```

// A compléter : appeler la méthode deleteProduit du service

```

}
}

```

Partie 6 : Keywords

Spring Data JPA permet de créer automatiquement des requêtes grâce aux **query methods**.

Les noms des méthodes doivent respecter certains **keywords** pour générer la requête SQL correspondante.

Exemples de keywords les plus utilisés :

- **findBy** → pour récupérer selon un attribut
- **findAllBy** → pour récupérer tous selon un critère
- **deleteBy** → pour supprimer selon un critère
- **countBy** → pour compter le nombre d'enregistrements
- **And, Or** → pour combiner plusieurs critères
- **OrderBy** → pour trier les résultats

Exemples de méthodes dans un repository :

```

@Repository
public interface UserRepository extends JpaRepository <User,Long> {
    no usages
    List<User> findByNom(String nom);
    no usages
    User findByEmailAndStatus(String email, boolean status);
    1 usage
    void deleteById(Long id);
    no usages
    List<User> findAllByRoleOrderByNomAsc(String role);
}

```

Partie 7 : JPQL

JPQL est un langage de requêtes orienté objet qui ressemble au SQL mais fonctionne avec les entités Java.

Caractéristiques :

- Utilise les noms des classes et des attributs, pas les tables et colonnes.
- Permet de faire des sélections, jointures, agrégations, conditions, etc.

```

@Repository
public interface UserRepository extends JpaRepository <User,Long> {
    no usages
    @Query("SELECT u FROM User u WHERE u.nomUser = :nom")
    List<User> findUsersByNom(@Param("nom") String nom);

    no usages
    @Query("DELETE FROM User u WHERE u.status = false")
    void deleteInactiveUsers();
}

```

Points clés :

@Query	Permet de définir une requête JPQL personnalisée.
:param	est utilisé pour passer les paramètres.

Partie 8 : Affectations

Qu'est-ce que l'affectation ?

- L'affectation consiste à lier des entités entre elles en respectant les relations définies (One-to-Many, Many-to-One, One-to-One, Many-to-Many).
- On attribue des objets « fils » à un objet « parent » ou vice-versa, selon le type de relation.

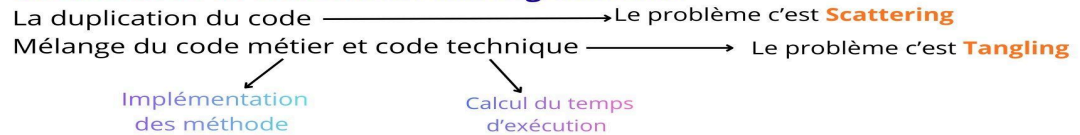
Exemple :

One To One Affectation DetailEquipe à Equipe Dans cet exemple, nous souhaitons enregistrer le DetailEquipe et l'affecter à l'Equipe associée déjà existante dans la base. Equipe est le parent donc l'affectation se fera sur l'objet Equipe Parent Equipe -idEquipe: Integer -nomEquipe: String -niveau: Niveau Child DetailEquipe -idDetailEquipe: Integer -salle: Integer -thematique: String 1 1	One To One Affectation DetailEquipe à Equipe <pre>public class Equipe implements Serializable { private static final long serialVersionUID = 1L; @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Integer idEquipe; private String nomEquipe; @Enumerated(EnumType.STRING) private Niveau niveau; @OneToOne private DetailEquipe detailEquipe; } public class DetailEquipe implements Serializable { private static final long serialVersionUID = 1L; @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Integer idDetailEquipe; private Integer salle; private String thematique; @OneToOne(mappedBy = "detailEquipe") @JsonIgnore private Equipe equipe; }</pre>
One To One Affectation DetailEquipe à Equipe <pre>@Override public Equipe affecterDetailEquipeToEquipe(DetailEquipe dt, Integer idEquipe) { Equipe equipe = equipeRepository.findById(idEquipe).get(); // equipe parent dans l'association donc affecter le child au parent // sauvegarder l'objet detail equipe dans la bd DetailEquipe detailEquipe = detailEquipeRepository.save(dt); equipe.setDetailEquipe(detailEquipe); // sauvegarder le nouveau état de l'objet avec le detail affecté equipeRepository.save(equipe); return equipe; }</pre>	One To One Affectation DetailEquipe à Equipe <pre>// http://localhost:8089/vadden/equipe/affecterDetailEquipeToEquipe/1 /* cette méthode permet d'affecter un détail équipe à son équipe */ @PostMapping("/{affectDetailEquipeToEquipe/{idEquipe}") @ResponseBody public Equipe affecterDetailEquipeToEquipe(@RequestBody DetailEquipe dt, @PathVariable("idEquipe") Integer idEquipe) { Equipe equipe = equipeService.affecterDetailEquipeToEquipe(dt, idEquipe); return equipe; }</pre>

Partie 9 : AOP

Exemple : Si je veux pour ChambreService :
Calculer les temps d'exécution pour chaque méthode ,
et pour chaque méthode j'écris hello méthode avant l'exécution

Les limites de ce Code ? : Cros Cutting Concerns



La solution ? :

Eviter la duplication du code —————> **Dry** : Don't repeat ur self
Séparation du code métier et technique —————> **Soc**: Séparation of concerns

Implémentation de L'AOP :

1. Aspect

- Une classe aspect est une classe annotée avec **@Aspect**.
- Elle est composée de Pointcuts et Advices.

2. Pointcut

- Un Pointcut est une expression qui permet de sélectionner un ensemble de méthodes.
- Exemple : `execution(public * tn.esprit.tpfoyer.service..*(..))`
- Il permet d'appliquer l'Aspect sur toutes les méthodes publiques d'un package précis.

3. Advice

- L'Advice est le code que l'on souhaite injecter.
- Il peut être exécuté avant, après ou autour de la méthode ciblée par le Pointcut.

4. JoinPoint

- Le JoinPoint est l'endroit précis où l'Aspect s'applique, généralement une méthode dans un service.

♦ Types d'Advice courants

1. @Before

Exécute l'Advice avant l'exécution de la méthode.

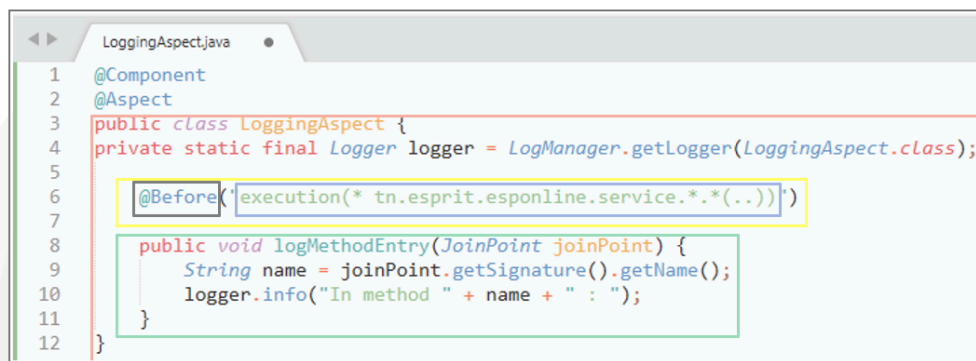
2. @After :Exécute l'Advice après l'exécution de la méthode.

3. **@Around** : Permet de contourner l'exécution de la méthode, et d'exécuter du code avant et après la méthode.
4. **@AfterThrowing** : Exécute l'Advice après qu'une exception a été lancée. Utile pour gérer les exceptions.
5. **@AfterReturning** : Exécute l'Advice après la fin normale de la méthode, lorsque la méthode retourne une valeur sans exception.

♦ Activation de l'AOP

- Pour activer Spring AOP, ajouter l'annotation **@EnableAspectJAutoProxy** dans la classe principale de l'application.

→ Indiquer quel est l'Aspect, le JoinPoint et le PointCut, l'Advice et le type d'advise :



```

1  @Component
2  @Aspect
3  public class LoggingAspect {
4      private static final Logger logger = LogManager.getLogger(LoggingAspect.class);
5
6      @Before("execution(* tn.esprit.esponline.service.*(..))")
7
8      public void logMethodEntry(JoinPoint joinPoint) {
9          String name = joinPoint.getSignature().getName();
10         logger.info("In method " + name + " : ");
11     }
12 }
  
```

Partie 10 : Scheduler

Le Scheduler permet d'exécuter automatiquement des tâches à des moments précis ou de manière répétée.

Exemples : automatisation d'envoi d'emails, notifications, rapports périodiques, etc.

♦ Activation du Scheduler

Pour utiliser le scheduler dans Spring Boot, ajouter l'annotation **@EnableScheduling** dans la classe principale.

Règles pour travailler avec un Scheduler

- La **méthode planifiée** doit :
 - Ne pas avoir de paramètres
 - Retourner **void**
- Ajouter l'annotation **@Scheduled** au-dessus de la méthode

- Définir la fréquence ou l'expression de planification

Types de planification

1. **fixedDelay**

- La tâche est exécutée immédiatement lors du démarrage.
- Spring attend que la tâche précédente se termine avant de relancer.

```
@Scheduled(fixedDelay = 5000) // en millisecondes
```

```
public void taskWithFixedDelay() {
```

```
    // code à exécuter
```

```
}
```

2. **fixedRate**

- La tâche est exécutée immédiatement lors du démarrage.
- Spring relance la tâche toutes les X secondes, même si la précédente n'est pas encore terminée.

```
@Scheduled(fixedRate = 5000) // en millisecondes
```

```
public void taskWithFixedRate() {
```

```
    // code à exécuter
```

```
}
```

3. **cron expression**

- Permet d'exécuter la tâche à un moment précis.
- L'expression comporte 5 parties : secondes, minutes, heures, jours du mois, mois, jours de la semaine.

```
@Scheduled(cron = "0 0 12 * * MON-FRI") // tous les jours de la semaine à 12:00
```

```
public void taskWithCron() {
```

```
    // code à exécuter
```

```
}
```

Partie 11: DTO

Qu'est-ce qu'un DTO ?

- DTO signifie Data Transfer Object.
- C'est un objet qui permet de transférer les données entre différentes couches d'une application.

Pourquoi utiliser un DTO ?

1. Sécurité :

On ne souhaite pas exposer toutes les informations de l'entité, par exemple les mots de passe.

2. Optimisation :

On envoie uniquement les informations nécessaires au client.

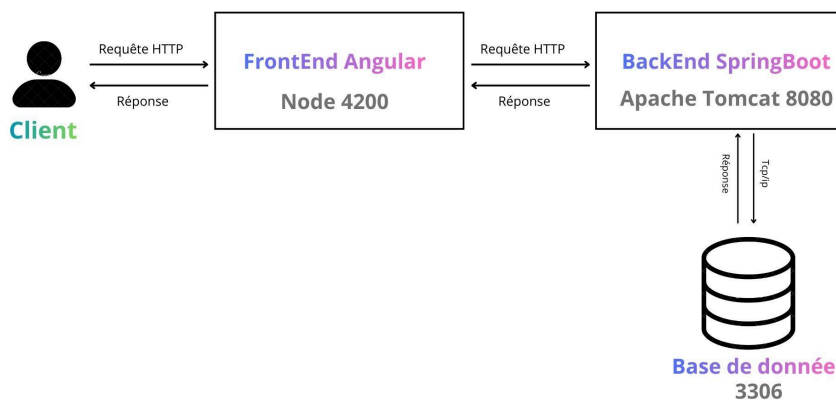
Comment créer un DTO ?

Il existe deux manières :

1. **Manuelle** : création d'une classe DTO avec les attributs nécessaires, getters/setters, et mappage manuel depuis l'entité.
2. **Automatique** : utilisation de bibliothèques comme **ModelMapper** ou **MapStruct** pour mapper automatiquement les entités vers les DTO et inversement.

Partie 12 : Architectures

Architecture Physique : N=4



Architecture Logique

